Short Notes — Design: Coupling & Cohesion

Page 1

Page 1: Definition of Software Design — converting requirements ('what') into design ('how'); design is creative.

Short Notes — Design: Coupling & Cohesion

Page 2

Page 2: Design overview — design concepts, process, methods, quality listed.

Short Notes — Design: Coupling & Cohesion

Page 3

Page 3: Intro to Design Concept (brief header).

Short Notes — Design: Coupling & Cohesion

Page 4

Page 4: Fundamental concepts — abstraction, refinement, modularity, architecture, procedure, hiding.

Short Notes — Design: Coupling & Cohesion

Page 5

Page 5: Data abstraction illustrated with a 'door' example and its attributes.

Short Notes — Design: Coupling & Cohesion

Page 6

Page 6: Procedure abstraction — separating 'open' action algorithm from object knowledge.

Short Notes — Design: Coupling & Cohesion

Page 7

Page 7: Library object-model example — classes: Library, Book, Copy, Patron; relationships: aggregation and loan roles.

Diagram (summarized): Recreated diagram summary: Library aggregates Books and Patrons; Book->Copy relation; Loan links Copy an

Short Notes — Design: Coupling & Cohesion

Page 8

Page 8: Continuation/diagram space for the library model (class diagram reference).

Short Notes — Design: Coupling & Cohesion

Page 9

Page 9: Class diagram of the library model (visual).

Short Notes — Design: Coupling & Cohesion

Page 10

Page 10: Evaluating borrow() abstraction — best when higher-level knows few details; delegate to lower-level classes.

Short Notes — Design: Coupling & Cohesion

Page 11

Page 11: Alternate borrow() design that exposes loan internals — poorer abstraction (anti-pattern).

Short Notes — Design: Coupling & Cohesion

Page 12

Page 12: Refinement example — stepwise algorithm for 'open' with loops, checks, key insertion.

Short Notes — Design: Coupling & Cohesion

Page 13

Page 13: Quote: Two ways to construct software design — make it simple or so complex no defects visible (C.A.R. Hoare).

Short Notes — Design: Coupling & Cohesion

Page 14

Page 14: Quote: Abstraction is fundamental to cope with complexity (G. Booch).

Short Notes — Design: Coupling & Cohesion

Page 15

Page 15: Benefits of modularity: easier to build, change, and fix.

Short Notes — Design: Coupling & Cohesion

Page 16

Page 16: Characterization of modules: simple, encapsulate data/operations, provide resources, encapsulated realization.

Short Notes — Design: Coupling & Cohesion

Page 17

Page 17: Module properties: replaceable, free of side-effects, testable, compilable & developable independently.

Short Notes — Design: Coupling & Cohesion

Page 18

Page 18: Advantages of modular systems: understanding, testing, portability, maintenance, reuse.

Short Notes — Design: Coupling & Cohesion

Page 19

Page 19: Measuring module quality: classic metrics (LOC, cyclomatic, Halstead) and more useful metrics (coupling/cohesion).

Short Notes — Design: Coupling & Cohesion

Page 20

Page 20: Divide and conquer concept and modular evaluation criteria (decomposability/composability/etc.).

Short Notes — Design: Coupling & Cohesion

Page 21

Page 21: Modularity trade-off — development vs integration cost; optimal module count idea.

Short Notes — Design: Coupling & Cohesion

Page 22

Page 22: Procedure design: focus on processing details, exact decisions, layered procedures.

Short Notes — Design: Coupling & Cohesion

Page 23

Page 23: Information hiding — module-controlled interface with hidden algorithm/data/resource policies.

Short Notes — Design: Coupling & Cohesion

Page 24

Page 24: Why hide information — limits side effects, reduces global impact, enforces interfaces, discourages globals.

Short Notes — Design: Coupling & Cohesion

Page 25

Page 25: Design process — multiple abstraction levels; overlapping stages from informal to finished design.

Short Notes — Design: Coupling & Cohesion

Page 26

Page 26: Design phases — data design, architectural design, interface design, procedural design.

Short Notes — Design: Coupling & Cohesion

Page 27

Page 27: Design methods listed: decision tables, ER, flowcharts, OOD, PDL, Petri nets, state charts.

Short Notes — Design: Coupling & Cohesion

Page 28

Page 28: Quote defending structured design techniques (Marvin Zelkowitz).

Short Notes — Design: Coupling & Cohesion

Page 29

Page 29: Enumerated design methods (reiteration and examples).

Short Notes — Design: Coupling & Cohesion

Page 30

Page 30: Fan-in vs fan-out concept illustrated with module graph; M9 fan-in=2 fan-out=4.

Short Notes — Design: Coupling & Cohesion

Page 31

Page 31: Fan-in/fan-out rules: minimize high fan-out, aim for more fan-in with depth.

Short Notes — Design: Coupling & Cohesion

Page 32

Page 32: Structured design origins — top-down, Stevens/Myers/Constantine, Yourdon/Constantine.

Short Notes — Design: Coupling & Cohesion

Page 33

Page 33: Structure charts — transform DFD into collection of modules; choose top-level often the central transform.

Short Notes — Design: Coupling & Cohesion

Page 34

Page 34: Data flow model and transforms mapping inputs to outputs; transform mapping diagram.

Diagram (summarized): Data flow mapping: inputs -> transforms -> outputs; pick central transform as top-level module.

Short Notes — Design: Coupling & Cohesion

Page 35

Page 35: Different flows: afferent, efferent, transform, coordinate — types of data flow.

Short Notes — Design: Coupling & Cohesion

Page 36

Page 36: Structure chart example (transaction/ATM-like): Get Card Info -> Verify -> Make Transaction -> Dispense Cash.

Diagram (summarized): Recreated structure chart summary: GetCardInfo -> Verify (Card, Bank, Password) -> MakeTransaction -> Disp

Short Notes — Design: Coupling & Cohesion

Page 37

Page 37: (Diagram continuation / structure chart details).

Short Notes — Design: Coupling & Cohesion

Page 38

Page 38: Transform analysis — inputs transformed into outputs; mapping DFD to structure chart controllers.

Short Notes — Design: Coupling & Cohesion

Page 39

Page 39: Transformation analysis diagram showing controllers for incoming/transform/outgoing flows.

Short Notes — Design: Coupling & Cohesion

Page 40

Page 40: Transformation analysis with data storage, terminators, and functions mapping.

Short Notes — Design: Coupling & Cohesion

Page 41

Page 41: Further transformation grouping — controllers and transform function relations.

Short Notes — Design: Coupling & Cohesion

Page 42

Page 42: Transaction analysis — DFD alternatives ('or' semantics), main controller with reception path & dispatcher.

Short Notes — Design: Coupling & Cohesion

Page 43

Page 43: Further transaction flow analysis — breaking alternatives into transaction centre and reception paths.

Short Notes — Design: Coupling & Cohesion

Page 44

Page 44: Structured design summary: systematic approach, supports functional decomposition, but limited data abstraction support.

Short Notes — Design: Coupling & Cohesion

Page 45

Page 45: (Summary continuation — pros/cons of structured design).

Short Notes — Design: Coupling & Cohesion

Page 46

Page 46: Design quality intro — elusive, depends on priorities; focus here on maintainability.

Short Notes — Design: Coupling & Cohesion

Page 47

Page 47: Component independence — benefits and measurement via coupling and cohesion (Yourdon 1978).

Short Notes — Design: Coupling & Cohesion

Page 48

Page 48: Cohesion defined — relatedness of internal module resources; higher cohesion localizes changes.

Short Notes — Design: Coupling & Cohesion

Page 49

Page 49: Cohesion types overview: Functional, Sequential, Communicational, Procedural, Temporal, Logical, Coincidental.

Short Notes — Design: Coupling & Cohesion

Page 50

Page 50: Cohesion levels explained — Coincidental, Logical, Temporal, Procedural (weak forms).

Short Notes — Design: Coupling & Cohesion

Page 51

Page 51: Cohesion levels continued — Communicational, Sequential, Functional, Object cohesion (strong forms).

Short Notes — Design: Coupling & Cohesion

Page 52

Page 52: Coincidental cohesion example — unrelated functions bundled for convenience.

Short Notes — Design: Coupling & Cohesion

Page 53

Page 53: Logical cohesion — grouping by class of activity (e.g., Read-All-Files regardless of usage).

Short Notes — Design: Coupling & Cohesion

Page 54

Page 54: Example of logical cohesion with CASE file-code reading records and incrementing counters.

Short Notes — Design: Coupling & Cohesion

Page 55

Page 55: Temporal cohesion — grouping tasks executed at same time (e.g., initialization tasks).

Short Notes — Design: Coupling & Cohesion

Page 56

Page 56: Procedural cohesion — elements follow a determined procedure/sequence (e.g., record loop totals).

Short Notes — Design: Coupling & Cohesion

Page 57

Page 57: Communicational cohesion — elements operate on the same central data (e.g., field validations).

Short Notes — Design: Coupling & Cohesion

Page 58

Page 58: Sequential cohesion — outputs of one element feed the next; chain dependency within module.

Short Notes — Design: Coupling & Cohesion

Page 59

Page 59: Functional cohesion — all elements contribute to a single function; module has single responsibility.

Short Notes — Design: Coupling & Cohesion

Page 60

Page 60: Cohesion visual ranking (low to high) and explanation of independence within a module.

Short Notes — Design: Coupling & Cohesion

Page 61

Page 61: Cohesion value formula: Number of internal links / Number of total methods/procedures.

Short Notes — Design: Coupling & Cohesion

Page 62

Page 62: Measuring cohesion — data dependencies and control dependencies; definitions and examples.

Short Notes — Design: Coupling & Cohesion

Page 63

Page 63: Program slicing — output slice finds statements affecting an output; input slice finds affected statements.

Short Notes — Design: Coupling & Cohesion

Page 64

Page 64: Slicing tokens: glue tokens (GT), superglue (ST), adhesiveness metric, example adhesion calculation.

Short Notes — Design: Coupling & Cohesion

Page 65

Page 65: Coupling defined — measure of information interchange; tight coupling bad, loose coupling good.

Short Notes — Design: Coupling & Cohesion

Page 66

Page 66: Visual levels of coupling: uncoupled, loosely coupled, highly coupled.

Short Notes — Design: Coupling & Cohesion

Page 67

Page 67: Module communication via global data — shared accessible variables and scope implications.

Short Notes — Design: Coupling & Cohesion

Page 68

Page 68: Module communication via local data — variables declared and used inside a module only.

Short Notes — Design: Coupling & Cohesion

Page 69

Page 69: Coupling depends on references, amount of data passed, and control information passed between modules.

Short Notes — Design: Coupling & Cohesion

Page 70

Page 70: Types of coupling: Content (worst), Common (global), External (files), explanation starts.

Short Notes — Design: Coupling & Cohesion

Page 71

Page 71: Common coupling example with variables V1,V2 modified by multiple modules.

Short Notes — Design: Coupling & Cohesion

Page 72

Page 72: More coupling types: Control coupling (passing flags), Stamp coupling (passing complex structs), Data coupling (passing simple data).

Short Notes — Design: Coupling & Cohesion

Page 73

Page 73: Coupling visual ranking (No direct -> Data -> Stamp -> Control -> Common -> Content) and independence measure.

Short Notes — Design: Coupling & Cohesion

Page 74

Page 74: Coupling metric: Coupling Value = Number of external links / Number of total modules.

Short Notes — Design: Coupling & Cohesion

Page 75

Page 75: How to uncouple modules: exchange minimal info, avoid passing unrelated data, limit control flags, hide data.

Short Notes — Design: Coupling & Cohesion

Page 76

Page 76: Coupling and inheritance: OO systems loosely coupled via messaging, but subclasses are coupled to superclasses.

Short Notes — Design: Coupling & Cohesion

Page 77

Page 77: Coupling/Cohesion summary: aim for high cohesion and low coupling for good modularisation.

Short Notes — Design: Coupling & Cohesion

Page 78

Page 78: Design quality attributes continued: Understandability, Adaptability, Tractability header.

Short Notes — Design: Coupling & Cohesion

Page 79

Page 79: Understandability — depends on cohesion, naming, documentation, complexity; complexity reduces understandability.

Short Notes — Design: Coupling & Cohesion

Page 80

Page 80: Adaptability — design adaptability criteria: loose coupling, documentation, clear design visibility, self-contained components.

Short Notes — Design: Coupling & Cohesion

Page 81

Page 81: Design tractability — concept and visual (object interaction vs decomposition levels) concluding the chapter.